

Week 8 • Git-flow, and pulling out the service layer

Technical Interaction Design • ITU • Autumn 2026

Built 22 September 2026 • commit 98b7cdc

Latest version: <https://itu-tid.github.io/lecture-notes-pdf/Week-08.pdf>

CONTENTS

Team Collaboration with Git Flow

- `main` always compiles

Finding the Components

- Start by drawing boxes on the app you can already see
- Two boxes if either half would be useful on its own
- Extracting a component is moving JSX out and naming what it needs

Patterns of Component Communication

- Data goes down, as props
- Events come back up, as callbacks
- Passing `setTodos` down gives every row power over the whole list

Code Organization

- A folder structure to start from

Component Extraction Cheat Sheet

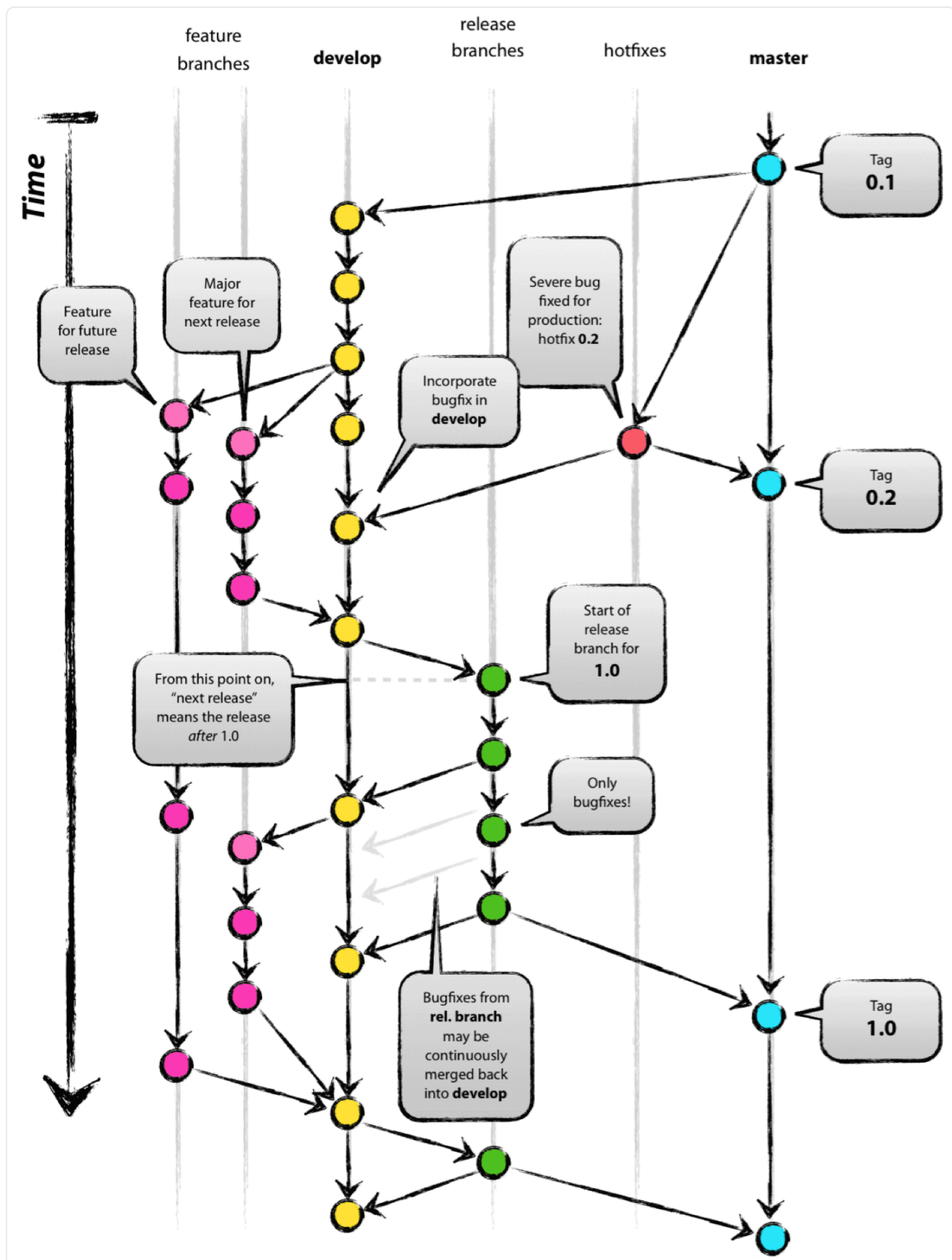
- Quick Decision Tree
- Bad and good, side by side
- The 4 Tests
- Step-by-Step Process
- Component Templates
- Common Mistakes
- Before/After Metrics
- Extraction checklist
- Pro Tips
- When Stuck
- Quick Links

Team Collaboration with Git Flow

Source: [Lectures/Technical/Tooling/Collaboration-with-Git-Flow.md](#) — something unclear? [Open an issue](#), or send a pull request.

In TID we use a collaboration model that's a modified version of a popular one named Git Flow.

Git Flow is a way of choreographing collaboration with Git that is designed around multiple types of branches as you can see in the following image:



In the course we make use of the following branches:

- **main** branch for releases / deliverables
- **Feature branches** for individual work

- the **development** branch for group integration

The **main** branch

- Contains releases
- Only modified by PR from develop
- Protected = only admins can accept PRs to it (TID staff are admins)

The **development** branch

- Contains the latest version of the team code
- Always ready for demo
- Point of integration of feature branches

Feature branches

- Are forked from **development**
- For independent / subgroup work
- Finished features are pushed to develop via Pull Request & tagging a team-member
 - Benefit 1: you get feedback
 - Benefit 2: everybody knows the code

The **.gitignore** file

- Specifies files that you don't want tracked
- Make sure that your `node_modules` folder is in **.gitignore**
- And that **.gitignore** is in the root of your repo folder

Exam Questions

1. Describe the three types of branches used in the course's Git Flow model.
2. Why should `node_modules` be in **.gitignore** ?

main always compiles

The one rule that is not a matter of taste: **code on main must run**. Every cohort has had at least one team whose **main** did not compile, usually for days, usually because a branch was merged in a hurry near a deadline.

It matters more than it looks. **main** is what a teammate branches from, what the examiner clones, and what your deployed app is built from — so a broken **main** is not one person's problem for an afternoon, it is everybody's problem until somebody notices. Which is the whole argument for branches: you break your own branch, not the one four other people are standing on.

Pull, build, and run *before* you merge, not after.

Something unclear in this section? [Open an issue](#).

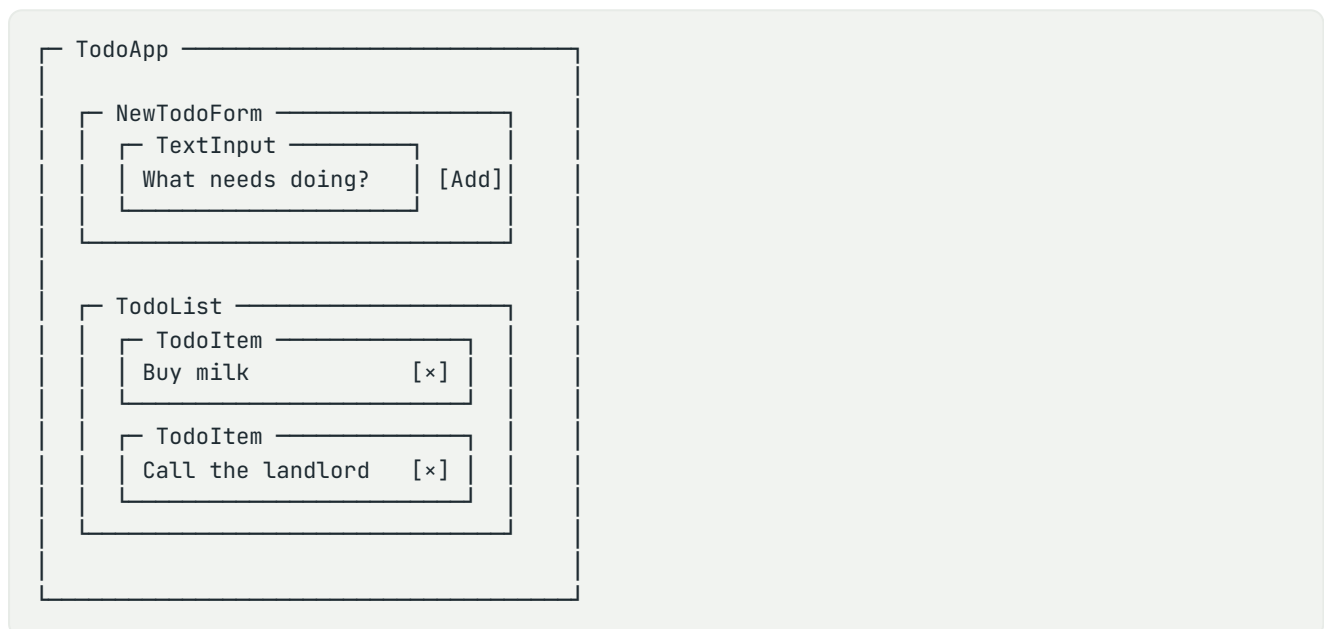
Finding the Components

Source: [Lectures/Technical/Structure/Finding-the-Components.md](#) — something unclear? [Open an issue](#), or send a pull request.

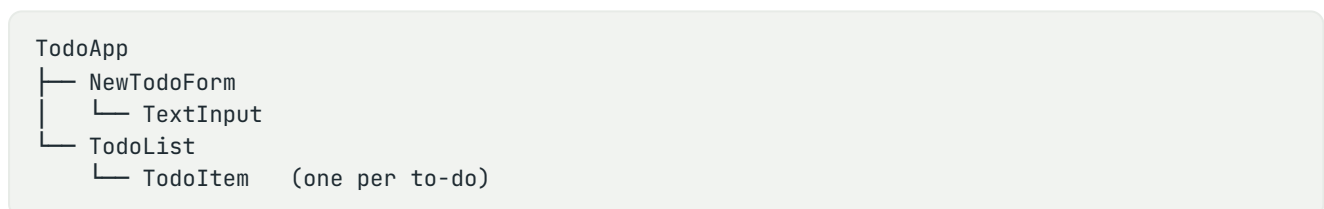
You know how to write a component. The harder question is which ones to write, and the answer is not obvious from the code, because at that point everything is already one big `return`.

So answer it on the screen instead. Draw boxes around the parts of the running interface, and name each box. It works both ways round: on a screen you have not built yet, and on one that already runs and has grown hard to read.

Start by drawing boxes on the app you can already see



Boxes inside boxes give you the hierarchy for free:



The labels are your names for the boxes, not anything the user sees. Boxes inside boxes give you the hierarchy for free.

Notice what is *not* a box. The **Add** button and the **x** are drawn but unnamed: each appears in exactly one place, inside one component. **Draw a box when the same thing appears more than once** — `TodoItem` once per to-do, `TextInput` wherever the app takes typing.

That is the stopping rule, and you need one, because without it you can keep subdividing until every `<span>` is a component. When you are unsure, leave it where it is: pulling a component out later is a five-minute job, and one you did not need is permanent clutter.

That is the whole exercise, and it takes two minutes on paper.

Something unclear in this section? Open an issue.

Two boxes if either half would be useful on its own

It is not about how many parts the thing has. `NewTodoForm` above is a text input *and* a button, and it is rightly one box: a button with nothing to submit is useless, and a field with no way to submit it is useless. Two parts, one job.

So the test is: **would either half be useful on its own? If yes, make them two. If neither means anything without the other, they are one.**

Then name it as a **thing, not an action**. A component is something on the screen, so it gets a noun: `TodoList`, `TodoItem`, `NewTodoForm`. Functions that *do* something get the verb: `handleAdd`, `handleRemove`. If you find yourself writing `AddTodo` as a component, you have named the button's job rather than the thing on the page.

- `TodoItem` — draws one to-do. One box.
- `TodoList` — draws all of them. One box.
- `NewTodoForm` — an input and a button, neither any use without the other. Still one box.

The common way to fail is welding a domain component to a layout one

Something that knows about to-dos, glued to something that would happily hold anything at all:

```
function TodoPanel({ firstName, children }) { // two components
  return (
    <>
      <h1>To Do List for {firstName}</h1>
      <div className="panel">{children}</div>
    </>
  );
}
```

Want the heading somewhere without a panel? Want a panel around something that is not a to-do list? You cannot have either, because they were glued together before anybody asked.

Both halves would be useful alone, and that is the whole argument. It is the exact opposite of `NewTodoForm`, where neither half would be. Notice too that the name gave nothing away: `TodoPanel` sounds like one thing.

Split, each half is usable on its own, and the caller says what it wants:

```
function Panel({ children }) {  
  return <div className="panel">{children}</div>;  
}  
  
// and at the call site – whatever goes between the tags arrives as `children`  
<Panel>  
  <h1>To Do List for {firstName}</h1>  
  <TodoList todos={todos} />  
</Panel>
```

Two boxes on the drawing, two components in the file.

Two other signals, both visible on the drawing rather than in the code:

It repeats. `TodoItem` appears once per to-do. Anything you would draw more than once is a component.

It matches the data. Notice that the tree above looks like the array it renders: a list of things, and a thing. When the component hierarchy and the data have the same shape, the code is usually easy; when they disagree, something is being forced.

Something unclear in this section? [Open an issue.](#)

Extracting a component is moving JSX out and naming what it needs

Here is the form before anything is pulled out of it, with the `<input>` from [Forms and Controlled Components](#), sitting inline:

```
function NewTodoForm({ onAdd }) {  
  const [text, setText] = useState("");  
  
  return (  
    <input  
      className="text-input"  
      value={text}  
      onChange={(e) => setText(e.target.value)}  
      placeholder="What needs doing?"  
    />  
  );  
}
```

`value` and `onChange` are ordinary props, so nothing stops you putting that input inside a component of your own. **Everything the moved JSX referenced from outside itself has to become a prop.** Here that is the value, the change handler, and the placeholder:

```
function TextInput({ value, onChange, placeholder }) {  
  return (  
    <input  
      className="text-input"  
      value={value}  
      onChange={(e) => onChange(e.target.value)}  
      placeholder={placeholder}  
    />  
  );  
}
```

One line inside `TextInput` does more than move: `onChange={(e) => onChange(e.target.value)}` unwraps the event and passes up a plain string. Write `onChange={onChange}` instead and the parent gets an event object where it expected text. That is the commonest way an extraction like this goes wrong.

The state stays where it was; `TextInput` has none of its own:

```
function NewTodoForm({ onAdd }) {
  const [text, setText] = useState("");

  return (
    <TextInput
      value={text}
      onChange={setText}
      placeholder="What needs doing?"
    />
  );
}
```

`useState` did not move into `TextInput`. If it had, `NewTodoForm` would have no idea what was typed, and could never clear the field or add the task. `TextInput` displays what it is given and reports what happened; it remembers nothing between renders.

Two things improved, and neither is about saving typing.

The parent stopped touching `e.target.value`. `TextInput` unwraps the event and hands up a plain string, so `NewTodoForm` can pass `setText` directly. The parent now works in the language of the app instead of the language of the DOM.

There is one place to change how inputs look. The `className`, and anything you add later, lives here rather than in every screen that happens to need typing.

And look at the shape of it: `TextInput` takes the value it should show, and a way to report that something changed. Exactly what the raw `<input>` takes. The pattern travelled up a level without changing, so the word travels with it: a **controlled component** is any component whose important state is held by its parent, exactly as a [controlled input](#) is an input whose value is held by the component around it.

Something unclear in this section? Open an issue.

Exam Questions

1. `NewTodoForm` holds an input and a button and is one component. `ToDoPanel` holds a heading and a container and should be two. What is the difference?
2. What is wrong with a component that renders a to-do heading *and* accepts arbitrary children ?
3. `TextInput` holds no state of its own. Why not, and what would break if it did?

References

- [Thinking in React](#) — the five steps, of which the first is the one worth keeping
- [Importing and Exporting Components](#) — once a component has its own name it usually wants its own file

Next: deciding *where the state lives* once you have pulled components out is its own problem, and extraction is what creates it: [Refactoring by Extracting Components](#).

Patterns of Component Communication

Source: [Lectures/Technical/React/Patterns-of-Component-Communication.md](#) — something unclear? [Open an issue](#), or send a pull request.

Components are arranged in a tree, and information has to travel along it. There are only two directions, and they use different mechanisms.

You have been writing both since week 2, without the names. The to-do list hands each row a `todo` and an `onRemove`; the row calls `onRemove` and knows nothing about what happens next. This note is that arrangement, named, and the shortcut that looks easier and is not.

Data goes down, as props

The parent hands the child what it needs to draw itself: for a row, the `todo` it should display. Data goes down.

Something unclear in this section? [Open an issue](#).

Events come back up, as callbacks

The child does not know who its parent is — and should not. Imagine a button: it has no idea who put it there, and it would be a worse button if it did.

What it does have is a function it was handed. Calling it is how the child says *something happened*, without knowing or caring what that means:

```
function TodoList() {
  const [todos, setTodos] = useState([]);

  function handleRemove(id) {
    setTodos(todos.filter((t) => t.id !== id));
  }

  return (
    <ul>
      {todos.map((todo) => (
        <TodoItem
          key={todo.id}
          todo={todo}           // down: what to draw
          onRemove={handleRemove} // up: a way to report a click
        />
      ))}
    </ul>
  );
}

function TodoItem({ todo, onRemove }) {
  return (
    <li>
      {todo.text}
      <button onClick={() => onRemove(todo.id)}>x</button>
    </li>
  );
}
```

Read the two directions:

- **Down, as props.** `TodoList` hands each row its `todo` — the data it needs to draw.
- **Up, as a callback.** `TodoItem` is handed `onRemove` and calls it. It does not know what happens next: it is not deleting anything, it is *reporting a click*.

`TodoItem` knows nothing about `filter`, about the array, or even that a list exists. Drop it into an app that archives items instead of deleting them, passing the same `onRemove`, and it works unchanged. That is what you get for keeping the arrow pointing one way.

Something unclear in this section? Open an issue.

Passing `setTodos` down gives every row power over the whole list

The tempting shortcut is to pass `setTodos` down and let the row remove itself:

```
<TodoItem todo={todo} todos={todos} setTodos={setTodos} /> // do not do this
```

It works. It also means every row now knows the whole list, and the shape of it, and how deletion is implemented — so changing any of those means changing the row too. You have swapped one small prop for three, and gained a component that only works in this app.

Hand down what the child needs to know, and a way to tell you something happened. Nothing else.

Next: once you extract components, two of them end up needing the same state, and neither owns it. That is *lifting state up*, and it is in [Refactoring by Extracting Components](#) — because extraction is what creates the problem in the first place.

Something unclear in this section? Open an issue.

Exam Questions

1. What are the two directions information travels between components, and what carries each?
2. `TodoItem` has a delete button but cannot delete anything. Why is that the right arrangement?
3. What goes wrong if you pass `setTodos` to every row instead of an `onRemove` callback?

Code Organization

Source: [Lectures/Technical/Structure/Code-Organization.md](#) — something unclear? [Open an issue](#), or send a pull request.

- If you implement a reusable component, put it in a /components folder
- Put screens in the /pages (or /screens) folder
- When you have more than one component to implement a bigger one, group them in a folder
- If you use styled components keep the associated style file adjacent to each component

A folder structure to start from

Separate Pages and Reusable Components

My favorite way of organizing

- **pages** – One file per page/view
- **components** – Reusable UI components
- **services** – API/backend logic
- **utilities** – a catch all for things that we don't know where to put yet

```
src/  
├─ assets/  
├─ pages/  
│   ├─ LoginPage.jsx  
│   └─ HomePage.jsx  
├─ components/  
│   ├─ TodoList/  
│   │   ├─ TodoList.jsx  
│   │   └─ TodoItem.jsx  
├─ services/  
│   ├─ authService.js  
│   └─ todoService.js  
├─ constants/  
├─ utilities/  
└─ App.jsx
```

Also, refactor, refactor, refactor. When you find a better organization, go with that.

Something unclear in this section? [Open an issue](#).

Component Extraction Cheat Sheet

Source: [Lectures/Technical/Structure/Component-Extraction-Cheatsheet.md](#) — something unclear? [Open an issue](#), or send a pull request.

On the examples below. They are written in TypeScript with Tailwind class names, because they came from a different course. This course is JavaScript and MUI. Read them for the *decisions* — when a block deserves a name, when repetition earns a component — and ignore the syntax around them; nothing here depends on the stack.

Quick Decision Tree

```
Is this JSX block hard to understand?
├ YES → Can you describe it in 1-2 words?
│   ├── YES → Extract component with that name
│   └ NO → Break it into smaller logical pieces
└ NO → Is it repeated 3+ times?
    ├── YES → Extract reusable component
    └ NO → Keep as is (don't over-abstract)
```

Bad and good, side by side

Layout Components

```
// Bad - What is this div?
<div className="mx-auto flex w-full max-w-xl flex-col gap-6 px-4 py-6">

// Good - Clear semantic name
<ProfileContainer>
```

Domain Components

```
// Bad - Implementation details everywhere
<section className="flex w-full flex-col items-center gap-2 p-6">
  <div className="size-24 rounded-3xl bg-neutral-300">
    <img src={user.avatar} />
  </div>
  <h2>{user.name}</h2>
</section>

// Good - Semantic hierarchy
<ProfileHeader>
  <Avatar user={user} />
  <UserName>{user.name}</UserName>
</ProfileHeader>
```

Repeated Patterns

```
// Bad - Copy-pasted 10 times
<button className="rounded-xl bg-neutral-200 px-4 py-2 hover:bg-neutral-300">
  Click me
</button>

// Good - Reusable component
<Button variant="secondary">Click me</Button>
```

The 4 Tests

Test	Question	Example
Tech Lead Test	Can a non-CSS person understand the structure?	<code><ProfileHeader></code> , not <code><div className="..."></code>
Semantic Test	Can you name it in 1-2 words?	<code><JobCard></code> , not <code><Component1></code>
DRY Test	Is pattern repeated 3+ times?	Extract if yes
SLA Test	Is everything at the same abstraction level?	Mixing <code><Header></code> and <code><div className="..."></code> is not

Step-by-Step Process

1. Identify Sections

// Draw boxes around logical groups

Header Area	→ ProfileHeader
About Section	→ ProfileSection
Experience List	→ ExperienceList
└ Experience Item	→ ExperienceEntry
└ Experience Item	→ ExperienceEntry

2. Name Components

- Use **nouns**: Button, Card, Avatar
- Be **specific**: ProfileHeader not Header
- Show **purpose**: PrimaryButton not BlueButton
- Use **domain terms**: JobListing not Item

3. Create Component Files

```
components/  
├ ui/                ← Reusable primitives  
│   ├── Button.tsx  
│   └── Avatar.tsx  
├ layout/            ← Layout containers  
│   └── PageContainer.tsx  
└ [domain]/          ← Domain components  
    └ ProfileHeader.tsx
```

4. Extract Props

```
// Identify what varies  
<Button  
  variant="primary"    // Style variation  
  size="large"         // Size variation  
  onClick={handleClick} // Behavior  
>  
  Click me             // Content  
</Button>
```

5. Refactor Page

```
// Before: 275 lines of mixed HTML/CSS
const Profile = () => {
  return <div className="...">...</div>
}

// After: 60 lines of semantic components
const Profile = () => {
  return (
    <ProfileContainer>
      <ProfileHeader user={user} />
      <ProfileSections user={user} />
    </ProfileContainer>
  );
};
```

Component Templates

Layout Component

```
interface ContainerProps {
  children: ReactNode;
}

export const Container = ({ children }: ContainerProps) => {
  return (
    <div className="[your-layout-classes]">
      {children}
    </div>
  );
};
```

UI Component (with variants)

```
interface ButtonProps {
  variant?: 'primary' | 'secondary';
  size?: 'small' | 'medium' | 'large';
  children: ReactNode;
  onClick?: () => void;
}

export const Button = ({
  variant = 'primary',
  size = 'medium',
  children,
  onClick
}: ButtonProps) => {
  return (
    <button className={getButtonClasses(variant, size)} onClick={onClick}>
      {children}
    </button>
  );
};
```

List Component

```
interface ListProps<T> {
  items: T[];
  renderItem: (item: T) => ReactNode;
  emptyMessage?: string;
}

export const List = <T extends { id: string }>({
  items,
  renderItem,
  emptyMessage = "No items"
}: ListProps<T>) => {
  if (items.length === 0) return <p>{emptyMessage}</p>;

  return (
    <ul>
      {items.map(item => (
        <li key={item.id}>{renderItem(item)}</li>
      ))}
    </ul>
  );
};
```

Polymorphic Component (semantic flexibility)

```
interface CardProps {
  children: ReactNode;
  as?: 'div' | 'article' | 'section';
}

export const Card = ({ children, as: Element = 'div' }: CardProps) => {
  return (
    <Element className="rounded-lg border p-4">
      {children}
    </Element>
  );
};

// Usage
<Card as="article">...</Card>
```

Common Mistakes

Mistake	Why Bad	Fix
<code><div className="container"></code> everywhere	Meaningless name	<code><ProfileContainer></code> , <code><JobContainer></code>
<code>Component1</code> , <code>Component2</code>	No semantic meaning	Name by purpose
50-line component inline	Hard to read/test	Extract to file
15 props	Too coupled	Break into smaller components
<code><BlueButton></code>	Names appearance	<code><PrimaryButton></code> (names purpose)
Mixing <code><Header></code> with <code><div className="..."></code>	Mixed abstraction	Keep consistent level

Before/After Metrics

Good refactoring achieves:

- 50-70% line reduction in page files
- No component > 150 lines
- 3-5 reusable components per page
- Clear component hierarchy (3-4 levels max)
- Reviewable without CSS knowledge

Extraction checklist

- ☐ Identify all pages in your app
 - ☐ For each page, list the semantic sections
 - ☐ Name each section as a component
 - ☐ Extract layout components (containers, wrappers)
 - ☐ Extract UI primitives (buttons, avatars, cards)
 - ☐ Extract domain components (profile header, job card)
 - ☐ Refactor pages to use components
 - ☐ Test: Can someone unfamiliar with your CSS understand structure?
 - ☐ Document components with JSDoc
-

Pro Tips

1. **Start with layout** - Extract containers first
 2. **Then UI primitives** - Buttons, avatars, inputs
 3. **Finally domain** - Profile headers, job cards
 4. **Name your props for what varies** - variant, size, content; not colour
 5. **Write README** - Explain component usage
 6. **Keep it simple** - Don't over-engineer
-

When Stuck

Ask yourself:

1. What does this component DO? (name it that)
2. Is someone seeing this for first time confused? (extract)
3. Would I copy-paste this elsewhere? (extract)
4. Can I test this alone? (if no, extract)

The Golden Rule:

"Code is read 10x more than it's written. Optimize for reading."

Quick Links

- [Full Guide](#)
- [Semantic JSX Article](#)
- [React Docs: Composition](#)

Something unclear in this section? Open an issue.